DATASCI 207 — ETF Volatility Dataset: Time-Series EDA

Goal: Provide a comprehensive, time-series-appropriate EDA notebook for the wrangled ETF volatility panel (`ticker × date`) with macro/risk features.

What this notebook emphasizes

- Panel integrity: unique `(ticker, date)`, consistent date parsing, monotone time ordering
- Time-series EDA: coverage/gaps, distributions, regimes, autocorrelation, cross-sectional behavior
- **Leakage/label alignment checks**: recompute log returns, trailing realized volatility, and forward realized volatility label
- Baselines to beat: persistence baselines using trailing volatility

> **Tip:** Run top-to-bottom once. Then rerun just the sections you tweak (paths, ticker selection, cutoffs).

```
# Purpose:
# - Import core libraries used throughout the notebook.
# - Keep dependencies light; optional sections (seaborn/statsmodels) degrade gracefully.

import os
import math
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

warnings.filterwarnings("ignore")

# Optional: nicer plots + correlation heatmaps
try:
    import seaborn as sns
    HAS_SEABORN = True
except Exception:
    HAS_SEABORN = False

# Optional: ACF/PACF style diagnostics
try:
    from statsmodels.tsa.stattools import acf
    HAS_STATSMODELS = True
except Exception:
    HAS_STATSMODELS = False

print("seaborn:", HAS_SEABORN, "| statsmodels:", HAS_STATSMODELS)
```

```
seaborn: True | statsmodels: True
```

1) Configuration

Set your input path(s) here.

- **CSV** is easiest (portable).
- **Parquet** is fastest and preserves dtypes (if you have `pyarrow` installed).

If you are in **Colab + Drive**, typical paths look like:

- `/content/drive/MyDrive/Colab Notebooks/vol_dataset_post_wrangle_021026.csv`

```
# Mount drive

from google.colab import drive
drive.mount("/content/drive")
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
# Purpose:
# - Centralize all settings in one place.

# ---- Input paths (edit as needed) ----
DATA_PATH_CSV = r"/content/drive/MyDrive/Colab Notebooks/vol_dataset_post_wrangle_021026.csv"   # <-- change if needed
```

```
DATA_PATH_PARQUET = None  # e.g., r"/content/drive/MyDrive/Colab Notebooks/vol_dataset_post_wrangle_021026.parquet"

USE_INPUT = "csv"   # "csv" or "parquet"

# ---- Key columns ----
KEY_COLS = ["ticker", "date"]
DATE_COL = "date"
TICKER_COL = "ticker"

# ---- Core financial columns ----
PRICE_COL = "adj_close"
VOLUME_COL = "volume"
RETURN_COL = "daily_log_return"

# ---- Target (label) ----
TARGET_COL = "forward_vol_5d_annual_decimel_calculated"

# ---- Volatility feature columns (if present) ----
TRAILING_VOL_COLS = [
    "trailing_vol_annual_decimel_5d_calculated",
    "trailing_vol_annual_decimel_20d_calculated",
    "trailing_vol_annual_decimel_5d_factset",
    "trailing_vol_annual_decimel_20d_factset",
]

# ---- Macro / risk feature columns (wide) ----
MACRO_COLS = [
    "US_3M_TB_YLD",
    "US_10Y_BOND_YLD",
    "NYGOLDS",
    "OIL_WTI_S",
    "VIX",
]

# ---- EDA choices ----
EDA_SAMPLE_TICKER = "SPY"
BIG_GAP_DAYS = 10   # flag gaps larger than this many calendar days inside a ticker
ANNUALIZATION = 252

# ---- Time-split defaults (edit for your project) ----
TRAIN_END = "2022-12-31"
VAL_END   = "2024-12-31"   # test = after VAL_END

# ---- Output ----
OUTPUT_DIR = r"/content/drive/MyDrive/Colab Notebooks/outputs"
os.makedirs(OUTPUT_DIR, exist_ok=True)
```

2) Load dataset + parse dates + canonical sort

```
# Purpose:
# - Load the panel dataset (CSV or Parquet).
# - Convert DATE_COL to datetime.
# - Canonical sort: ticker ascending, date ascending (oldest → newest).

def load_dataset():
    if USE_INPUT.lower() == "csv":
        assert DATA_PATH_CSV is not None, "Set DATA_PATH_CSV"
        df_raw = pd.read_csv(DATA_PATH_CSV)
    elif USE_INPUT.lower() == "parquet":
        assert DATA_PATH_PARQUET is not None, "Set DATA_PATH_PARQUET"
        df_raw = pd.read_parquet(DATA_PATH_PARQUET)
    else:
        raise ValueError("USE_INPUT must be 'csv' or 'parquet'")
    return df_raw

df_raw = load_dataset()

print("Loaded shape:", df_raw.shape)
print("Columns:", len(df_raw.columns))

df = df_raw.copy()

# Parse dates robustly
df[DATE_COL] = pd.to_datetime(df[DATE_COL], errors="coerce")
```

```
# Fail-fast if parsing broke
bad_dates = df[DATE_COL].isna().sum()
print("Bad date parses:", bad_dates)
assert bad_dates == 0, "Some dates failed to parse. Inspect df[df[DATE_COL].isna()]."

# Canonical sort for all time-series operations
df = df.sort_values([TICKER_COL, DATE_COL], ascending=[True, True]).reset_index(drop=True)

print("Date range:", df[DATE_COL].min().date(), "→", df[DATE_COL].max().date())
df.head()
```

```
Loaded shape: (96525, 23)
Columns: 23
Bad date parses: 0
Date range: 2014-12-31 → 2025-12-31
```

	date	ticker	adj_close	has_price	volume_k	volume	trailing_vol_daily_pct_5d_factset	trailing_vol_daily_pct_20d_factset
0	2021-04-21	BEDZ	24.95	1.0	84.476	84476.0	1.5753	1.5827
1	2021-04-22	BEDZ	25.05	1.0	15.443	15443.0	1.5753	1.5827
2	2021-04-23	BEDZ	25.32	1.0	11.732	11732.0	1.5760	1.5833
3	2021-04-26	BEDZ	25.43	1.0	33.758	33758.0	1.5764	1.5837
4	2021-04-27	BEDZ	25.64	1.0	29.076	29076.0	1.5770	1.5843

5 rows × 23 columns

3) Panel integrity checks (keys, duplicates, monotone dates, coverage)

```
# Purpose:
# - Validate unique (ticker, date).
# - Check monotone dates within each ticker.
# - Summarize coverage.

# Duplicate key check
dup_count = df.duplicated(KEY_COLS).sum()
print("Duplicate (ticker,date) rows:", int(dup_count))

# Monotone date check within ticker
def count_non_monotone_dates(d):
    # d is already sorted by (ticker,date) globally, but verify per ticker
    return (~d.groupby(TICKER_COL)[DATE_COL].apply(lambda s: s.is_monotonic_increasing)).sum()

non_mono = int(count_non_monotone_dates(df))
print("Tickers with non-monotone dates:", non_mono)

# Coverage per ticker
coverage = (
    df.groupby(TICKER_COL)[DATE_COL]
      .agg(start="min", end="max", n_days="count")
      .sort_values("n_days", ascending=False)
)

print("Number of tickers:", coverage.shape[0])
display(coverage.head(10))
display(coverage.tail(10))
```

```
Duplicate (ticker,date) rows: 0
Tickers with non-monotone dates: 0
Number of tickers: 36
```

ticker	start	end	n_days
KBE	2014-12-31	2025-12-31	2767
IYZ	2014-12-31	2025-12-31	2767
KIE	2014-12-31	2025-12-31	2767
KCE	2014-12-31	2025-12-31	2767
XSD	2014-12-31	2025-12-31	2767
XME	2014-12-31	2025-12-31	2767
KRE	2014-12-31	2025-12-31	2767
PEJ	2014-12-31	2025-12-31	2767
PNQI	2014-12-31	2025-12-31	2767
PXQ	2014-12-31	2025-12-31	2767

ticker	start	end	n_days
XLV	2014-12-31	2025-12-31	2767
XLY	2014-12-31	2025-12-31	2767
XTN	2014-12-31	2025-12-31	2767
XPH	2014-12-31	2025-12-31	2767
XOP	2014-12-31	2025-12-31	2767
VGT	2015-01-02	2025-12-31	2766
XLRE	2015-10-08	2025-12-31	2573
FTXG	2016-09-21	2025-12-31	2333
XLC	2018-06-19	2025-12-31	1895
BEDZ	2021-04-21	2025-12-31	1181

4) Gaps and missingness (time continuity + feature missing rates)

```
# Purpose:
# - Identify big gaps in each ticker's date sequence (calendar-day gaps).
# - Summarize missingness by column and by ticker.

# Big gaps detection (calendar days)
tmp = df[[TICKER_COL, DATE_COL]].copy()
tmp["prev_date"] = tmp.groupby(TICKER_COL)[DATE_COL].shift(1)
tmp["gap_days"] = (tmp[DATE_COL] - tmp["prev_date"]).dt.days

big_gaps = tmp[tmp["gap_days"] > BIG_GAP_DAYS].copy()
print("Big gaps (gap_days >", BIG_GAP_DAYS, "):", big_gaps.shape[0])
display(big_gaps.head(20))

# Missingness by column
miss = df.isna().mean().sort_values(ascending=False)
miss_tbl = pd.DataFrame({"missing_rate": miss, "missing_count": df.isna().sum()})
display(miss_tbl.head(15))

# Missingness by ticker for the target (often informative)
if TARGET_COL in df.columns:
    tgt_miss_by_ticker = df.groupby(TICKER_COL)[TARGET_COL].apply(lambda s: s.isna().mean()).sort_values(ascending=False)
    display(tgt_miss_by_ticker.head(10))
```

```
Big gaps (gap_days > 10 ): 0
   ticker  date  prev_date  gap_days
```

	missing_rate	missing_count
NYGOLDS	0.000000	0
OIL_WTI_S	0.000000	0
US_10Y_BOND_YLD	0.000000	0
US_3M_TB_YLD	0.000000	0
VIX	0.000000	0
adj_close	0.000000	0
daily_log_return	0.000373	36
date	0.000000	0
forward_vol_5d_annual_decimel_calculated	0.001865	180
group_type	0.000000	0
has_price	0.000000	0
industry	0.390272	37671
market	0.971334	93758
sector	0.638394	61621
ticker	0.000000	0

ticker	forward_vol_5d_annual_decimel_calculated
BEDZ	0.004234
XLC	0.002639
FTXG	0.002143
XLRE	0.001943
VGT	0.001808
IYZ	0.001807
KBE	0.001807
KRE	0.001807
PEJ	0.001807
PNQI	0.001807

dtype: float64

5) Macro feature sanity (should be constant across tickers on a given date)

```
# Purpose:
# - On any given date, macro columns should not vary by ticker (they are shared market series).
# - We test this by counting unique values per date for each macro column.

present_macro = [c for c in MACRO_COLS if c in df.columns]
print("Macro columns present:", present_macro)

macro_uniques = {}
for c in present_macro:
    # number of unique values across tickers for each date
    nun = df.groupby(DATE_COL)[c].nunique(dropna=False)
    macro_uniques[c] = {
        "median_unique_per_date": float(nun.median()),
        "max_unique_per_date": int(nun.max()),
        "pct_dates_unique_gt_1": float((nun > 1).mean()),
    }

macro_check = pd.DataFrame(macro_uniques).T.sort_values("pct_dates_unique_gt_1", ascending=False)
display(macro_check)
```

```
# Interpretation:
# - Ideally: median_unique_per_date == 1 and pct_dates_unique_gt_1 == 0
```

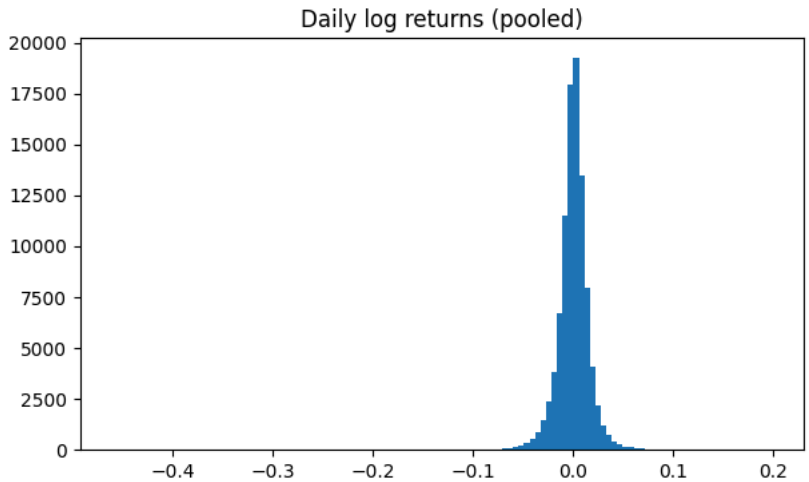
```
Macro columns present: ['US_3M_TB_YLD', 'US_10Y_BOND_YLD', 'NYGOLDS', 'OIL_WTI_S', 'VIX']
```

	median_unique_per_date	max_unique_per_date	pct_dates_unique_gt_1
US_3M_TB_YLD	1.0	1.0	0.0
US_10Y_BOND_YLD	1.0	1.0	0.0
NYGOLDS	1.0	1.0	0.0
OIL_WTI_S	1.0	1.0	0.0
VIX	1.0	1.0	0.0

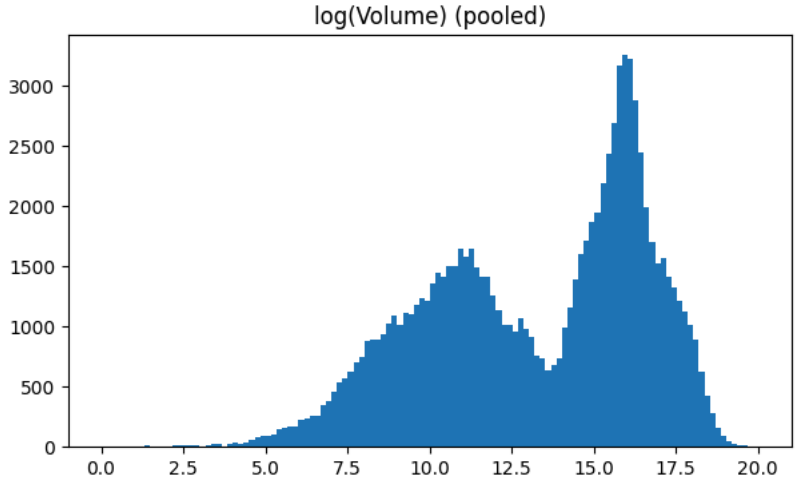
Next steps: Generate code with macro_check New interactive sheet

6) Distribution EDA (returns, volume, trailing vol, target vol)

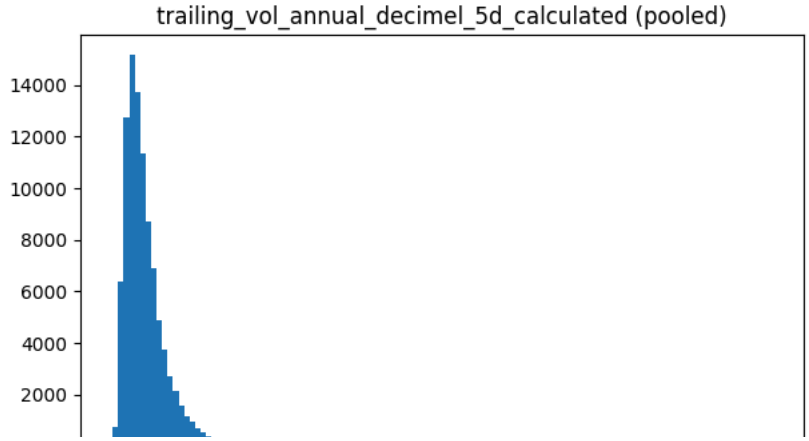
```
# Purpose:
# - Visualize distributions: returns, log(volume), target vol, trailing vol.
# - This gives you tail behavior, skew, and scaling needs.

def safe_hist(series, title, bins=100):
    s = series.dropna()
    plt.figure(figsize=(7,4))
    plt.hist(s, bins=bins)
    plt.title(title)
    plt.show()

# Returns
if RETURN_COL in df.columns:
    safe_hist(df[RETURN_COL], "Daily log returns (pooled)", bins=120)
    print(df[RETURN_COL].describe(percentiles=[0.01,0.05,0.5,0.95,0.99]))

# Volume
if VOLUME_COL in df.columns:
    vol = df[VOLUME_COL].replace(0, np.nan)
    safe_hist(np.log(vol), "log(Volume) (pooled)", bins=120)
    print(vol.describe(percentiles=[0.01,0.05,0.5,0.95,0.99]))

# Trailing vol examples
for c in [x for x in TRAILING_VOL_COLS if x in df.columns]:
    safe_hist(df[c], f"{c} (pooled)", bins=120)
    print(c, df[c].describe(percentiles=[0.01,0.05,0.5,0.95,0.99]).to_dict())

# Target
if TARGET_COL in df.columns:
    safe_hist(df[TARGET_COL], f"Target: {TARGET_COL} (pooled)", bins=120)
    print(df[TARGET_COL].describe(percentiles=[0.01,0.05,0.5,0.95,0.99]))
```

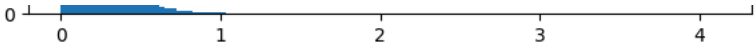



```
count    96489.000000
mean         0.000290
std          0.016145
min         -0.459984
1%          -0.044887
5%          -0.024899
50%          0.000772
95%          0.023346
99%          0.042107
max          0.197543
Name: daily_log_return, dtype: float64
```

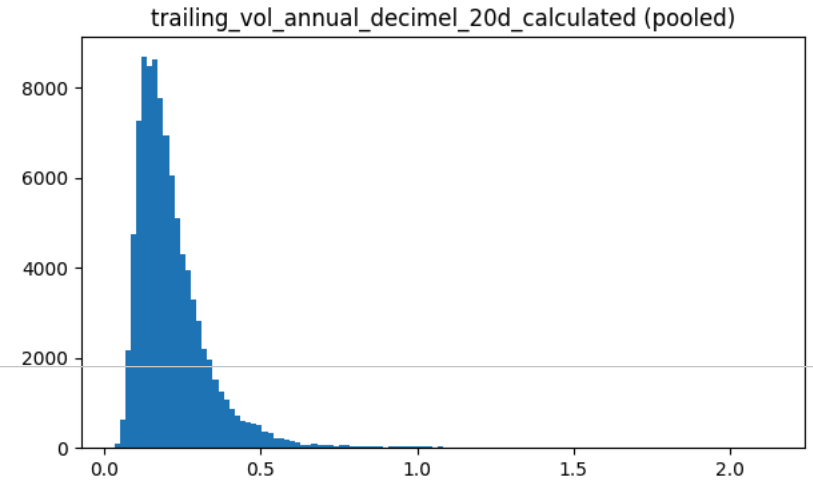



```
count    9.597800e+04
mean     9.397789e+06
std      2.049194e+07
min      1.000000e+00
1%       2.690000e+02
5%       1.915000e+03
50%      1.328570e+06
95%      4.827411e+07
99%      9.794572e+07
max      5.072443e+08
Name: volume, dtype: float64
```

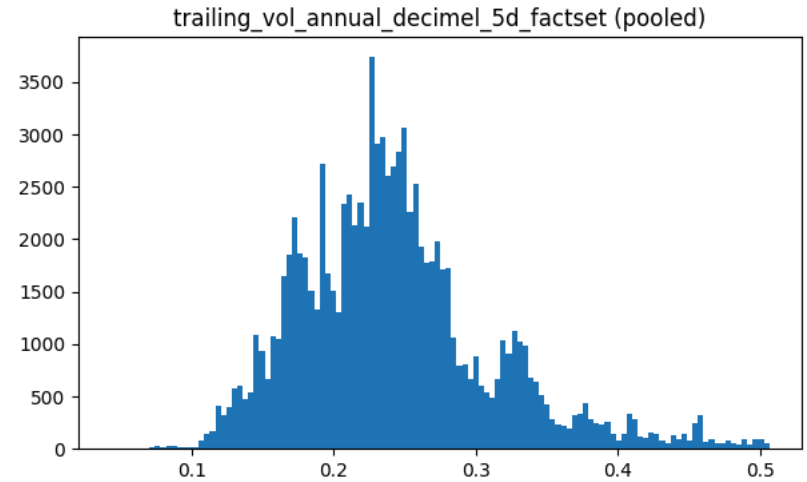




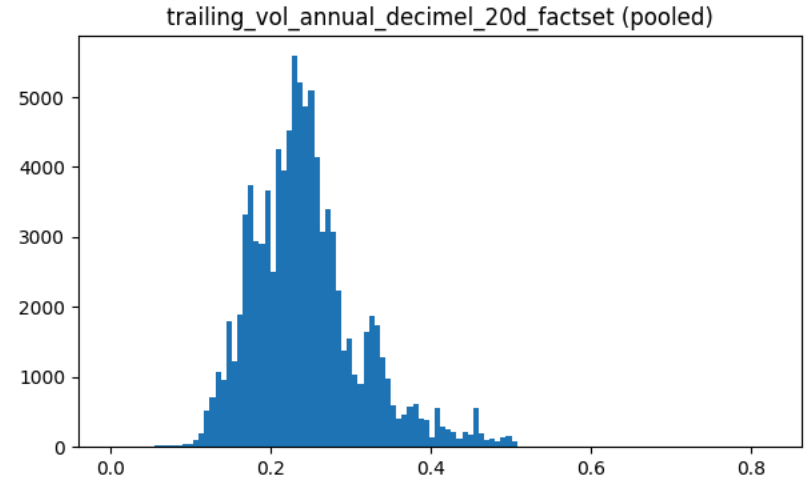

```
trailing_vol_annual_decimel_5d_calculated {'count': 96345.0, 'mean': 0.2069985458508485, 'std': 0.15347008081123947, 'min': 0.(
```



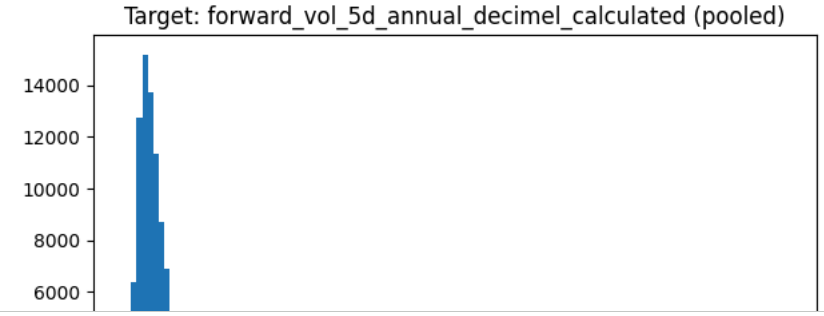

```
trailing_vol_annual_decimel_20d_calculated {'count': 95805.0, 'mean': 0.2201050028704138, 'std': 0.1324635772389395, 'min': 0.(
```




```
trailing_vol_annual_decimel_5d_factset {'count': 96525.0, 'mean': 0.24293003367003368, 'std': 0.06871785874580691, 'min': 0.04
```



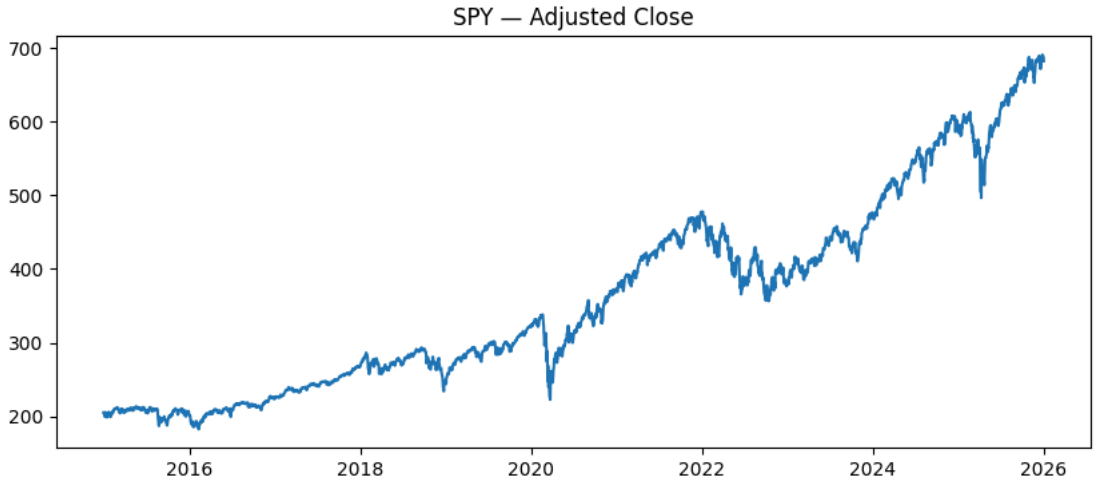

```
trailing_vol_annual_decimel_20d_factset {'count': 96489.0, 'mean': 0.24402550549803598, 'std': 0.06884942270723506, 'min': 0.0(
```

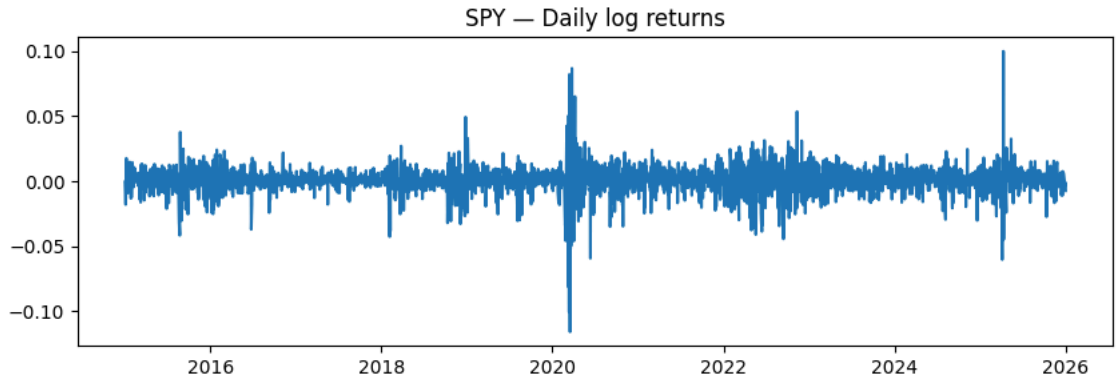


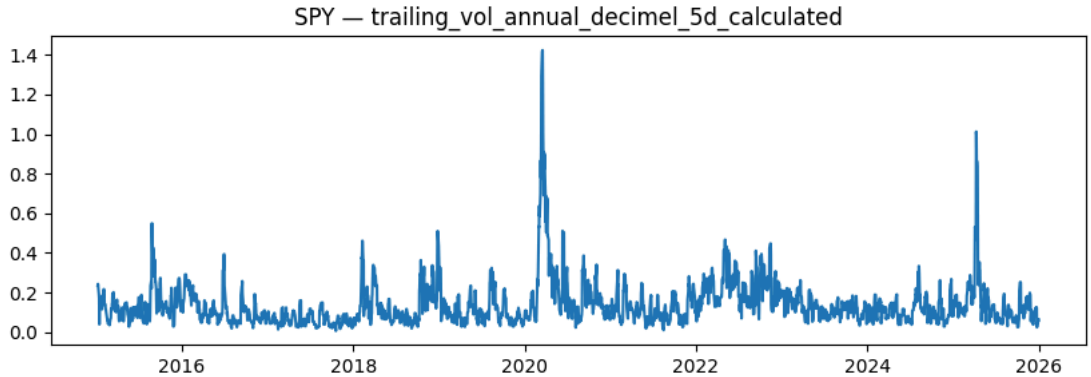
7) Time-series plots for one ticker (price, returns, trailing vol, forward vol)

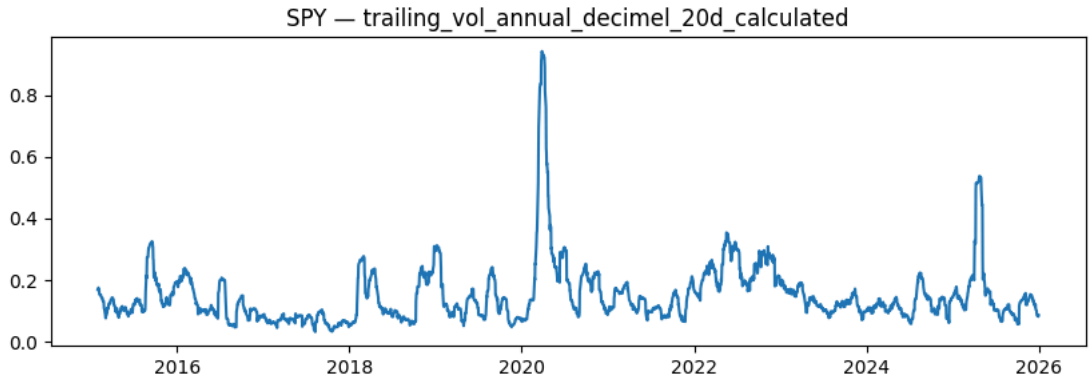
```
# Purpose:
# - Plot core series for a representative ticker.
# - Reveals volatility clustering and regime shifts.

ticker = EDA_SAMPLE_TICKER
d = df[df[TICKER_COL] == ticker].copy()

print("Ticker:", ticker, "| rows:", d.shape[0], "| range:", d[DATE_COL].min().date(), "→", d[DATE_COL].max().date())

# Price
if PRICE_COL in d.columns:
    plt.figure(figsize=(10,4))
    plt.plot(d[DATE_COL], d[PRICE_COL])
    plt.title(f"{ticker} — Adjusted Close")
    plt.show()

# Returns
if RETURN_COL in d.columns:
    plt.figure(figsize=(10,3))
    plt.plot(d[DATE_COL], d[RETURN_COL])
    plt.title(f"{ticker} — Daily log returns")
    plt.show()

# Trailing vol
for c in [x for x in TRAILING_VOL_COLS if x in d.columns]:
    plt.figure(figsize=(10,3))
    plt.plot(d[DATE_COL], d[c])
    plt.title(f"{ticker} — {c}")
    plt.show()

# Target forward vol
if TARGET_COL in d.columns:
    plt.figure(figsize=(10,3))
    plt.plot(d[DATE_COL], d[TARGET_COL])
    plt.title(f"{ticker} — Target (forward 5d realized vol, annualized)")
    plt.show()

# Macro series overlay (optional)
present_macro = [c for c in MACRO_COLS if c in d.columns]
for c in present_macro:
    plt.figure(figsize=(10,3))
    plt.plot(d[DATE_COL], d[c])
    plt.title(f"Macro: {c}")
    plt.show()
```

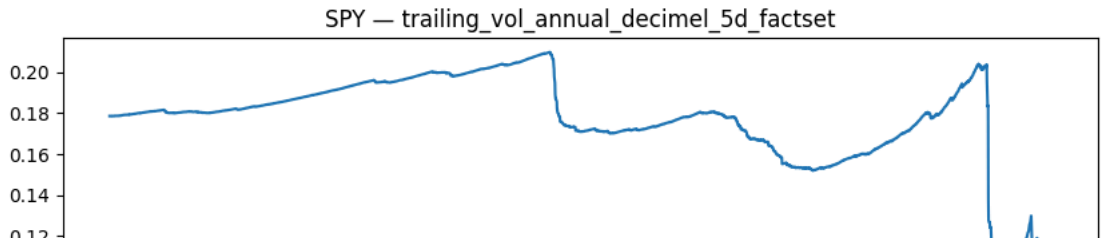
```
Ticker: SPY | rows: 2767 | range: 2014-12-31 → 2025-12-31
```

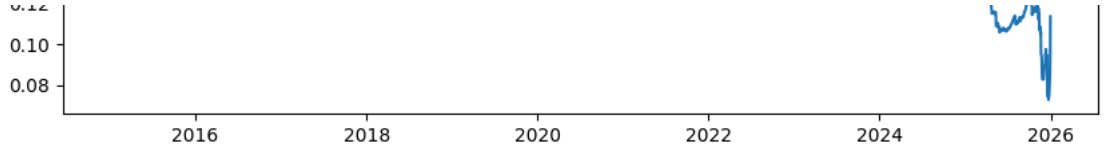


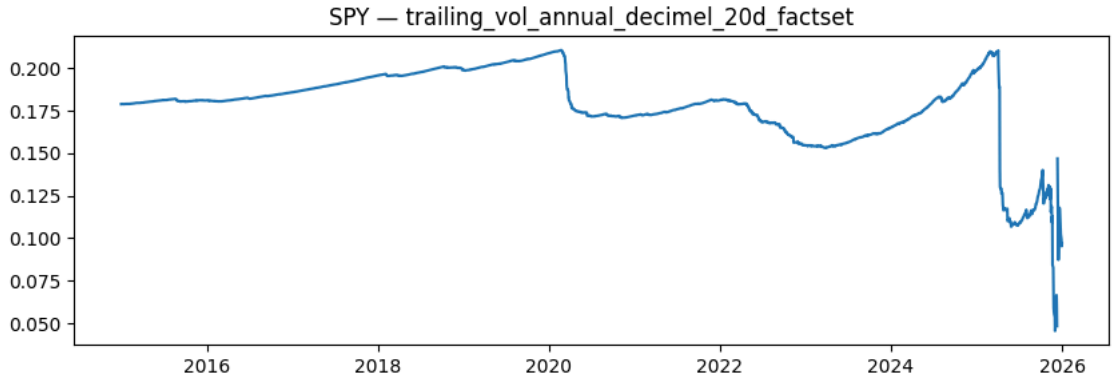


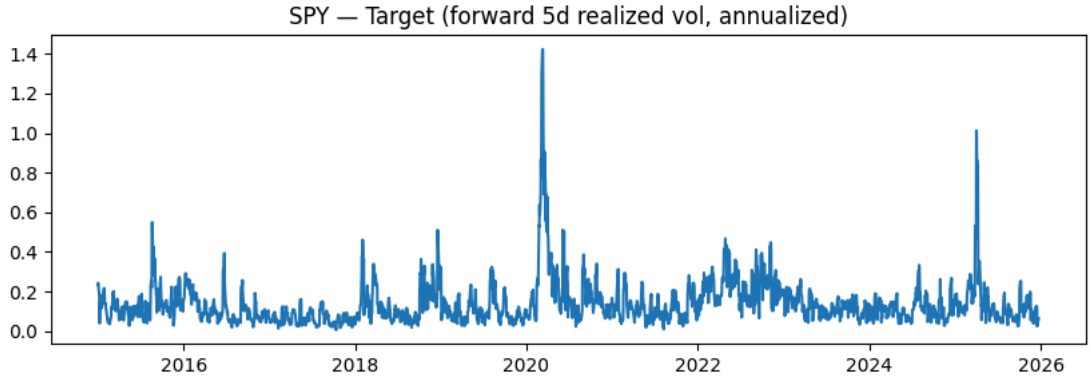


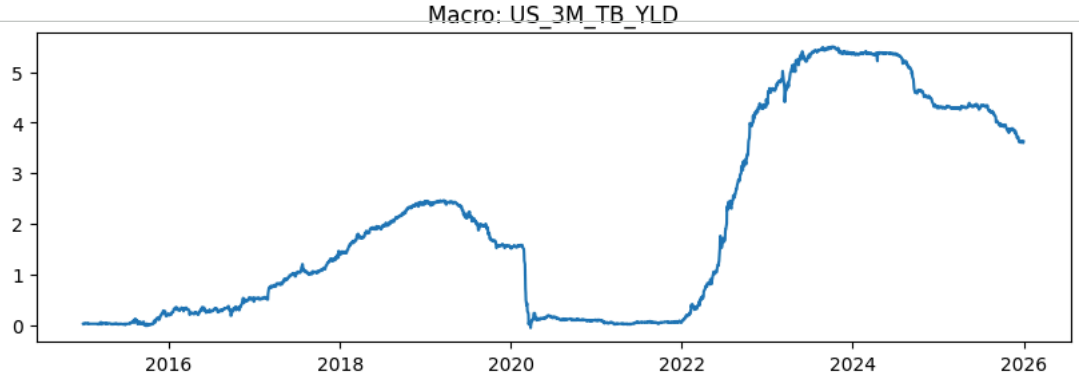


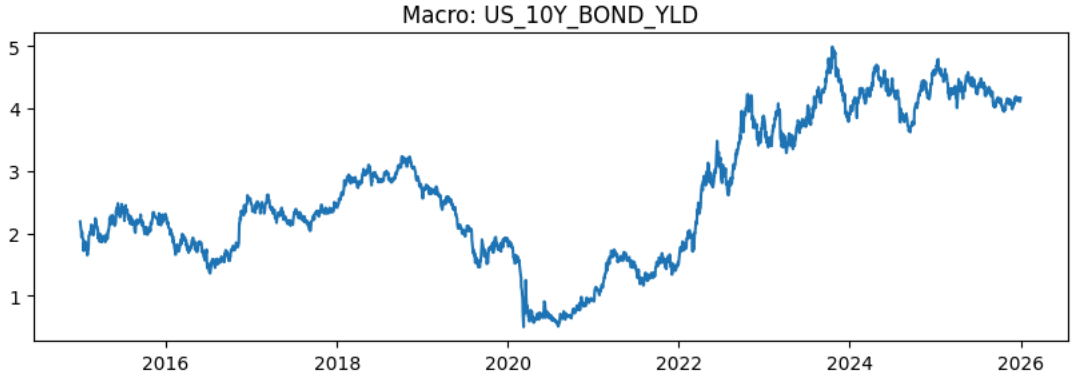



0.10
0.08
2016
2018
2020
2022
2024
2026


SPY — trailing_vol_annual_decimel_20d_factset
0.200
0.175
0.150
0.125
0.100
0.075
0.050
2016
2018
2020
2022
2024
2026


SPY — Target (forward 5d realized vol, annualized)
1.4
1.2
1.0
0.8
0.6
0.4
0.2
0.0
2016
2018
2020
2022
2024
2026


Macro: US_3M_TB_YLD
5
4
3
2
1
0
2016
2018
2020
2022
2024
2026


Macro: US_10Y_BOND_YLD
5
4
3
2
1
2016
2018
2020
2022
2024
2026

Macro: NYGOLDS
4500
4000
3500
3000
2500